UNIVERSITÀ DEGLI STUDI DI MILANO BICOCCA

Fitting a Neural Network

Lecture 13

Course of:

Statistics and data analysis

Federico De Guio - Martino Borsato

Outline of today's lecture

- (Stochastic) gradient descent
- Backpropagation
- Implementation and training issues
- Network optimization
- Dealing with incomplete or imbalanced datasets

Optimization algorithms: (stochastic) **gradient descent**

- James, cap.10
- Prince, cap.6

Optimizing the learning objective

- The **optimizer** is the algorithm that decides how to **adjust the network weights in order to reduce the error**. It answers two questions:
  - What direction should I take?
  - How large my step should be?

- In supervised learning, we want to optimize the objective

$$l(w) = \sum_{i=1}^{N} L(y_i, f(x_i \mid w)) \longrightarrow$$

Total distance between function and the full target dataset

- We need an **optimization algorithm** to find the **optimal** (or just "**good**") *w*

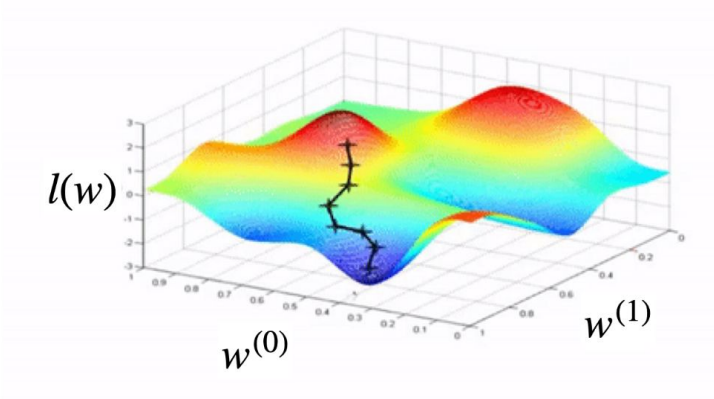
Gradient descent

- Set *w(t = 0)* to some values (e.g., *w(0) = 0* or some random value)

- At iteration t:
  - Compute the gradient $\nabla_w l(w(t))$ → direction of **steepest increase** of *l(w)* at *w(t)*
  - Take a small step in the **opposite direction**:

$$w(t+1) = w(t) - \eta \nabla_w l(w(t))$$

Step size / learning rate



Gradient descent interpretation for NN

- Move in the direction of maximum steepness towards a smaller cost value
  - Hard to digest in many dimensions

- In practice: gives indications about what weights are **the ones making the steepness change more**
  - **How sensitive the cost function is to weights and bias**

Linear regression with gradient descent

- **Learning objective**:

$$l(w) = \sum_{i=1}^{N} (y_i - w^\intercal x_i)^2$$

Dot product → it's our **model prediction**

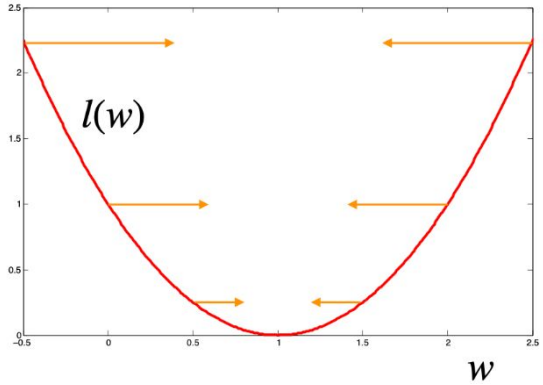
$$w^\top x_i = \sum_{j=1}^{d} w_j x_{i,j} = w_1 x_{i,1} + w_2 x_{i,2} + \cdots + w_d x_{i,d}$$

- **Partial derivatives**:

$$\frac{\partial}{\partial w^{(j)}} l(w) = \frac{\partial}{\partial w^{(j)}} \sum_{i=1}^{N} (y_i - w^\intercal x_i)^2 = -2 \sum_{i=1}^{N} (y_i - w^\intercal x_i) x_i^{(j)}$$

- **Gradient descent update**:

$$w(t+1) = w(t) + 2\eta \sum_{i=1}^{N} (y_i - w^\intercal x_i) x_i$$



Limitation of gradient descent

- Requires a **full pass** over the training dataset at each iteration:

$$w(t+1) = w(t) - \eta \nabla_w \underbrace{\sum_{i=1}^{N} L(y_i, f(x_i, | w))}_{l(w)}$$

- **Prohibitively expensive** if the training dataset is large!
  - Deep networks have millions or billions of weights → **we can can't naively compute all gradients independently!**

Stochastic gradient descent - the mathematical justification

- The learning objective decomposes additively:

$$l(w) = \frac{1}{N}\sum_{i=1}^{N} L(y_i, f(x_i, |\, w)) = \mathbb{E}_{(x,y)\in S}\left[L(y, f(x\,|\,w)\right]$$

add normalization

This is the **expectation value** of the cost function for a single event randomly picked from the dataset S

- The total gradient is the expected gradient of the single-example losses:

$$\nabla_w l(w) = \nabla_w \mathbb{E}_{(x,y)\in S}\left[L(y, f(x\,|\,w)\right] = \mathbb{E}_{(x,y)\in S}\left[\nabla_w L(y, f(x\,|\,w)\right]$$

- Gradient descent update: $w(t+1) = w(t) - \eta \nabla_w l(w(t))$
- SGD update: $w(t+1) = w(t) - \eta \nabla_w L(y, f(x\,|\,w(t)))$

Unbiased estimate of $\nabla_w l(w(t))$

for a random $(x, y) \in S$

Stochastic gradient descent

- SGD is an **online** optimization algorithm (only needs one example at a time)

- In practice, instead of using single events, we use **mini-batch SGD** → compute gradient on a mini-batch of training example (e.g., 8, 16, or 32 examples)
  - Leverages fast vector operations (especially on GPU)
  - Decreases volatility of gradient updates (but there is still some useful noise)
  - Can be parallelized (e.g. different cores compute gradients on different mini-batches)
  - Useful also for least-squares regression on large datasets

- Note: **no need to check validation error at every iteration**

SGD practical tips

- Divide the loss function by the number of examples (normalize):

$$w(t+1) = w(t) - \frac{\eta}{N} \nabla_w l(w)$$

(Don't want the size of our updates to depend on the N)

- Start with a large step size $\eta(t=0)$
- Whenever the validation error stops going down, lower the step size:

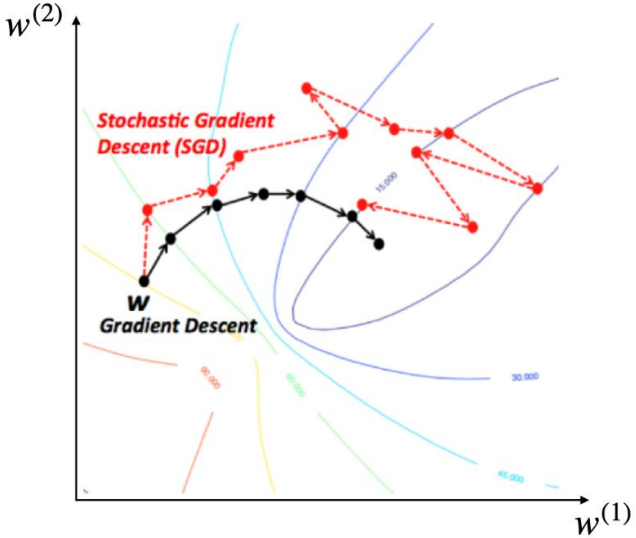
$$\eta(t+1) = \eta(t)/2$$

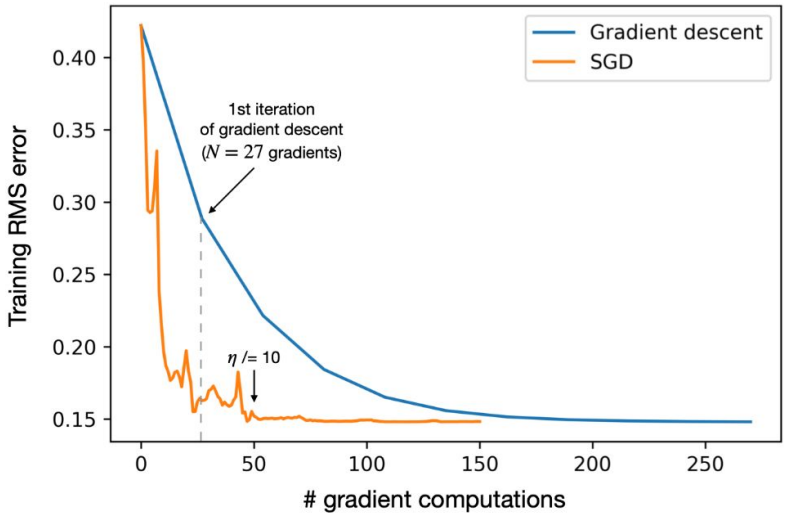
(step size must decrease over time to guarantee convergence)

- Stop when the validation error no longer decreases (early stopping)

GD vs SGD

- Gradient descent update: $w(t+1) = w(t) - \eta \nabla_w l(w(t))$
- SGD update: $w(t+1) = w(t) - \eta \nabla_w L(y, f(x \mid w(t)))$
  for a random $(x, y) \in S$





SGD requires less gradient computations

Backpropagation

How much each weight in the network is to be blamed for the total error signal?

- James, cap.10
- Prince, cap.7

The **chain rule**

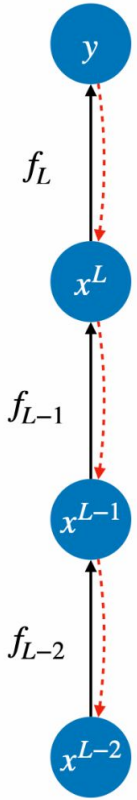
- Up to now we've seen how to minimize the error for a single neuron → expand to **multi-layer net**

$$y = f_L(\ldots f_2(f_1(x))\ldots)$$

$$\begin{aligned} y &= f_L(x_L) \\ x_L &= f_{L-1}(x_{L-1}) \\ &\vdots \\ x_1 &= f_1(x) \end{aligned}$$

We want to compute $\frac{\partial y}{\partial x_l}$ for all $l \in \{1,\ldots,L\}$

Imagine a net with a single neuron per layer



$$\frac{\partial y}{\partial x_L}$$

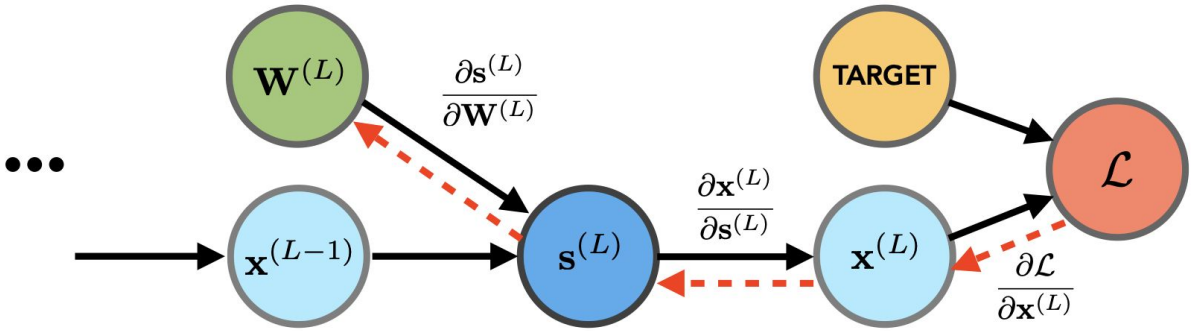
$$\frac{\partial y}{\partial x_{L-1}} = \frac{\partial y}{\partial x_L}\frac{\partial x_L}{\partial x_{L-1}}$$

$$\frac{\partial y}{\partial x_{L-2}} = \frac{\partial y}{\partial x_{L-1}}\frac{\partial x_{L-1}}{\partial x^{L-2}}$$

$$\frac{\partial y}{\partial x_{L-3}} = \frac{\partial y}{\partial x_{L-2}}\frac{\partial x_{L-2}}{\partial x^{L-3}}$$

At each step we can reuse the computation of the previous step!

Backpropagation for NNs



$\frac{\partial \mathcal{L}}{\partial \mathbf{x}^{(L)}}$	How much does the Loss change wrt the output?
$\frac{\partial \mathbf{x}^{(L)}}{\partial \mathbf{s}^{(L)}}$	How much does the output change wrt the activation function?
$\frac{\partial \mathbf{s}^{(L)}}{\partial \mathbf{W}^{(L)}}$	How much does the activation function change wrt the weight?

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(L)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{x}^{(L)}} \frac{\partial \mathbf{x}^{(L)}}{\partial \mathbf{s}^{(L)}} \frac{\partial \mathbf{s}^{(L)}}{\partial \mathbf{W}^{(L)}}$$

depends on the form of the loss

derivative of the non-linearity

$$\frac{\partial}{\partial \mathbf{W}^{(L)}} (\mathbf{W}^{(L)\intercal} \mathbf{x}^{(L-1)}) = \mathbf{x}^{(L-1)\intercal}$$

note $\nabla_{\mathbf{W}^{(L)}} \mathcal{L} \equiv \frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(L)}}$ is notational convention

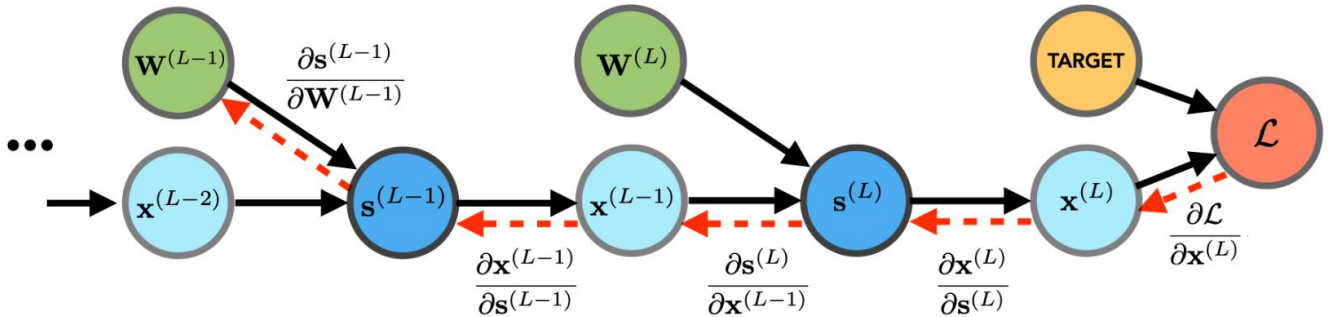
Backpropagation for NNs

Note: we can reuse previous calculations!

now let's go back one more layer...

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(L)}} = \boxed{\frac{\partial \mathcal{L}}{\partial \mathbf{x}^{(L)}} \frac{\partial \mathbf{x}^{(L)}}{\partial \mathbf{s}^{(L)}}} \frac{\partial \mathbf{s}^{(L)}}{\partial \mathbf{W}^{(L)}}$$

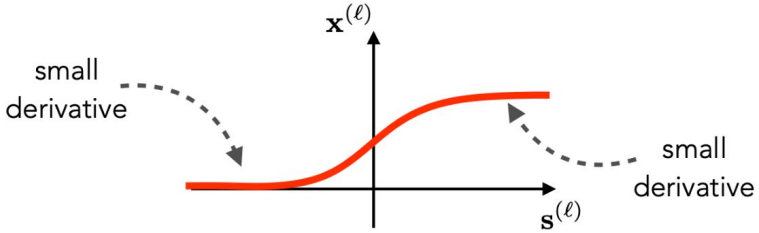
again we'll draw the dependency graph:

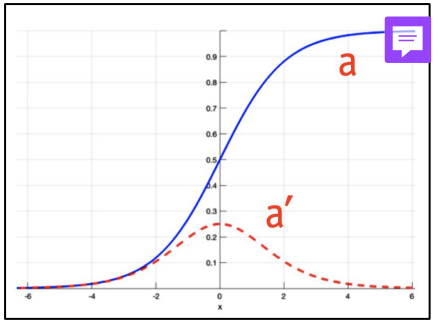


$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(L-1)}} = \boxed{\frac{\partial \mathcal{L}}{\partial \mathbf{x}^{(L)}} \frac{\partial \mathbf{x}^{(L)}}{\partial \mathbf{s}^{(L)}}} \frac{\partial \mathbf{s}^{(L)}}{\partial \mathbf{x}^{(L-1)}} \frac{\partial \mathbf{x}^{(L-1)}}{\partial \mathbf{s}^{(L-1)}} \frac{\partial \mathbf{s}^{(L-1)}}{\partial \mathbf{W}^{(L-1)}}$$

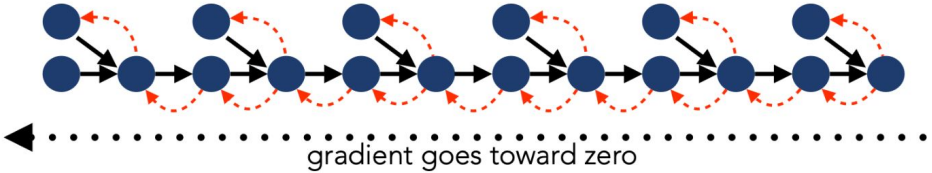
Implementation and training issues

The vanishing gradient problem





saturating non-linearities have *small* derivatives almost everywhere

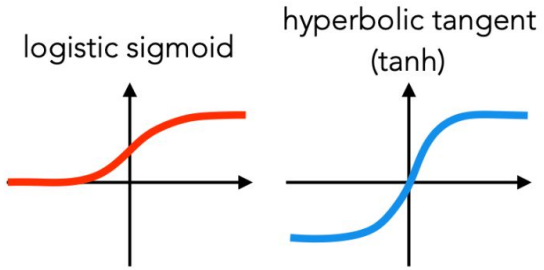


in backprop, the product of many small terms (i.e. $\frac{\partial \mathbf{x}^{(\ell)}}{\partial \mathbf{s}^{(\ell)}}$) goes to zero

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(\ell)}} = \cdots \frac{\partial \mathbf{x}^{(L)}}{\partial \mathbf{s}^{(L)}} \cdots \frac{\partial \mathbf{x}^{(L-1)}}{\partial \mathbf{s}^{(L-1)}} \cdots \frac{\partial \mathbf{x}^{(\ell+1)}}{\partial \mathbf{s}^{(\ell+1)}} \cdots \frac{\partial \mathbf{x}^{(\ell)}}{\partial \mathbf{s}^{(\ell)}} \frac{\partial \mathbf{s}^{(\ell)}}{\partial \mathbf{W}^{(\ell)}}$$

difficult to train very deep networks with saturating non-linearities

Nonlinearities

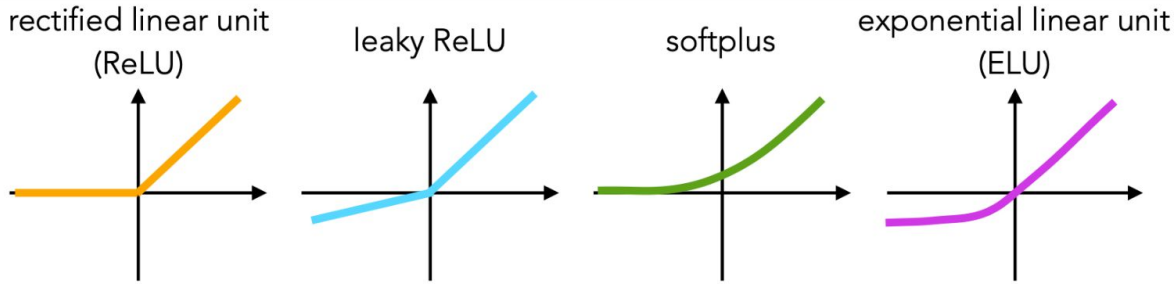


saturating

derivative goes to

zero at +∞ and -∞





non-saturating

non-zero derivative

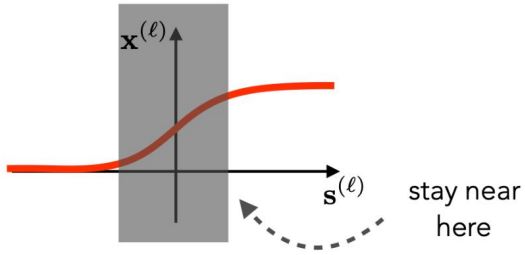
at +∞ and/or -∞

most often used

$$\text{ReLU}(x) := \max(0, x)$$

Batch normalization

→ keep the inputs within the dynamic range of the non-linearity

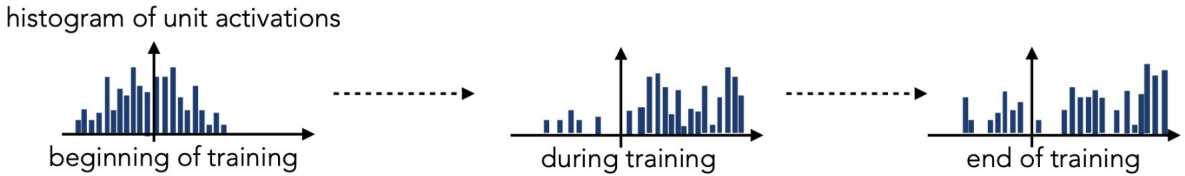


we can ***normalize*** the activations before applying the non-linearity

$$\mathbf{s} \leftarrow \frac{\mathbf{s} - \text{shift}}{\text{scale}}$$

Why is batch normal needed?

- While the weight of the first layers change, the distribution of the data arriving to the internal layers also changes → this is called **internal covariate shift**
  - Deep layers need to keep changing to adapt to the changes of the preceding layers → **learning become slow**

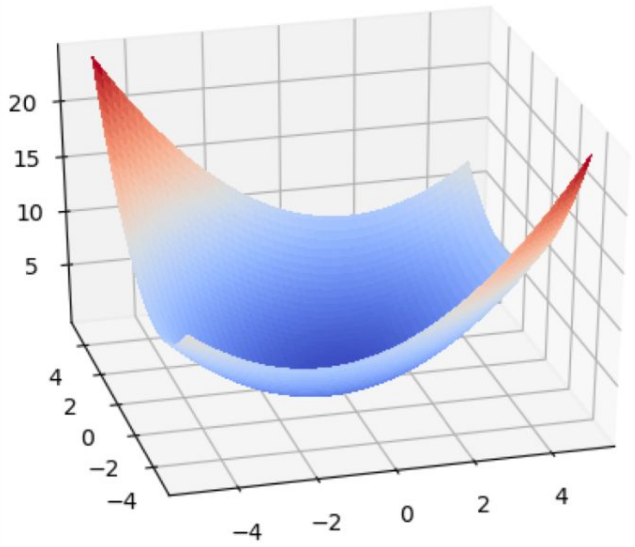


- **Batch normalization is supposed to stabilize the activations during training**

Loss landscape of NNs

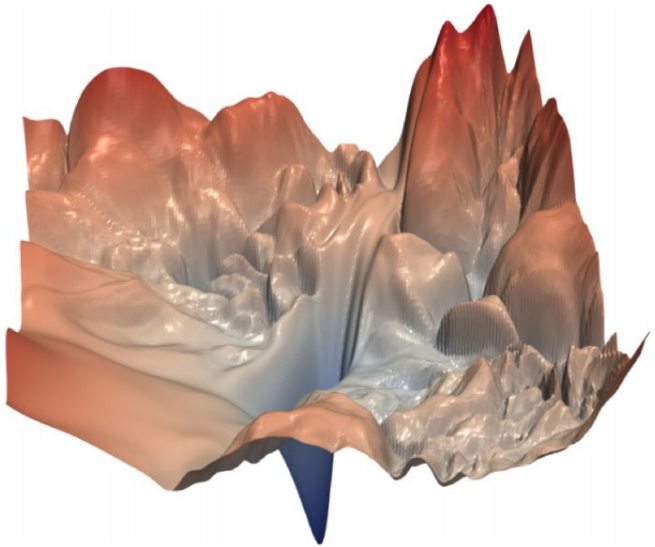
Convex problem
(logistic regression, SVMs)

$L(w)$



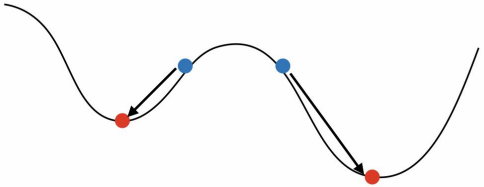
Deep Networks

$L(w)$

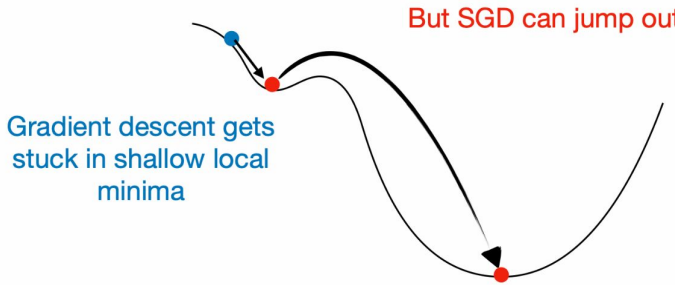


Consequences of **nonconvexity**

- **Sensitivity to initialization**: based on where you start you may end up in different minima



- **Shallow minima**: we may get stuck in a suboptimal local minimum



The noise of stochastic gradient descent is actually a benefit in deep learning!

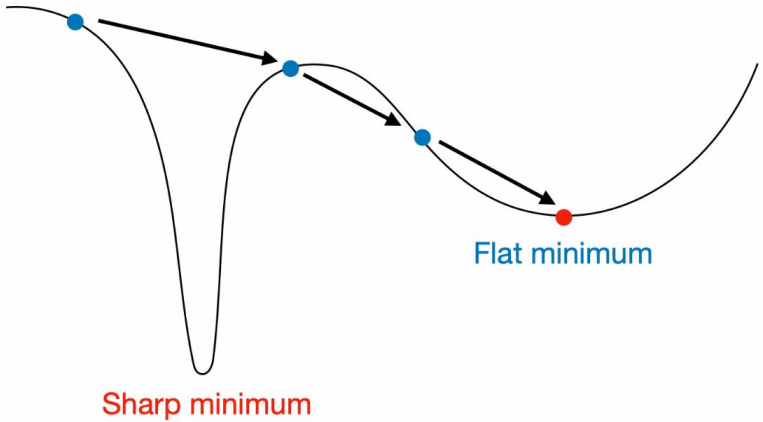
Flat and sharp minima

To converge to a minimum we need:

$$\eta < \frac{2}{\text{curvature}}$$

The noise of SGD makes us jump out of sharp minima

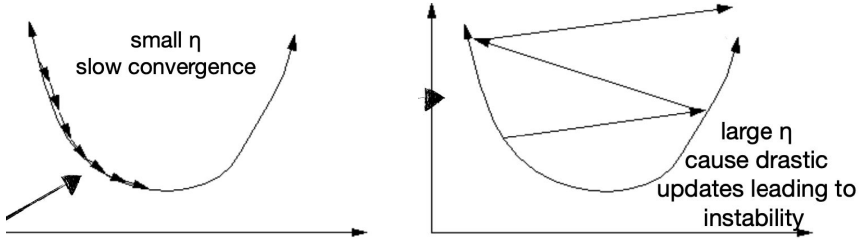
If the learning rate is too large we can't enter a sharp minimum



- Is this a problem? In deep learning it is often observed that **flat minima are better solutions**, so avoiding sharp minima is good!

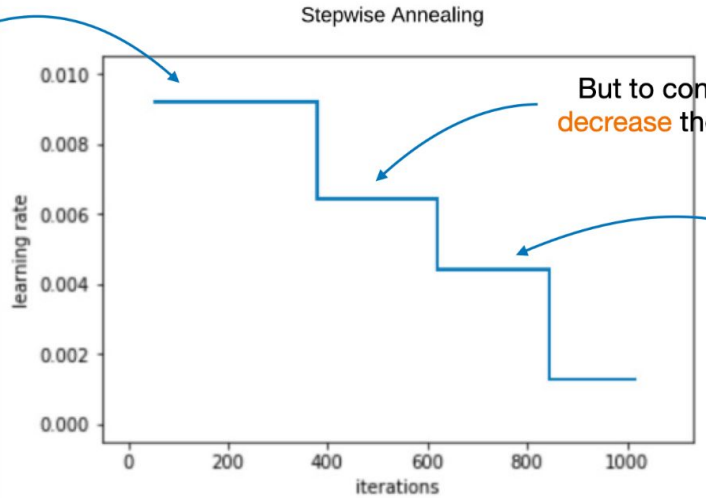
Learning rate annealing

- η affects the **speed and quality of convergence** toward a minimum



We start with an high learning rate

Converges faster and avoids sharp minima



But to converge we need to decrease the learning rate later

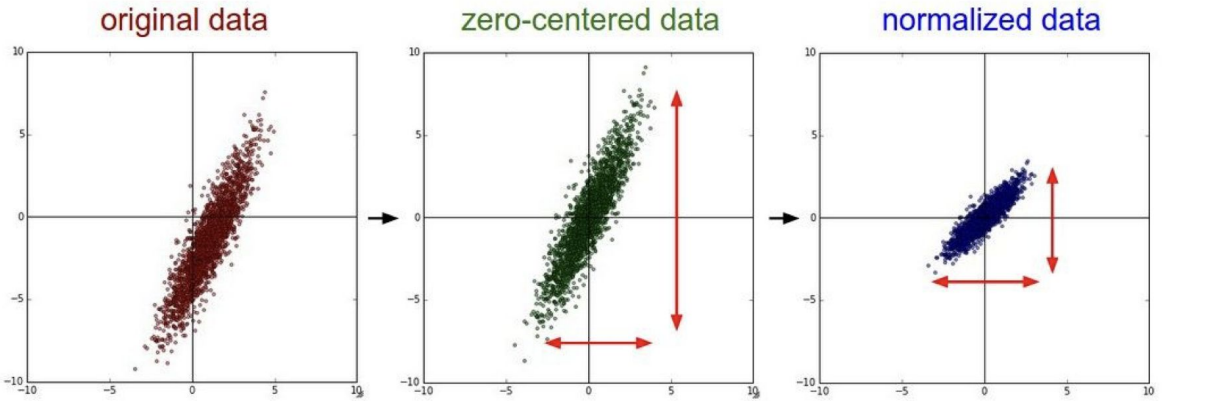
If we decrease too fast we end up in a bad minimum, so we do it in multiple steps

Network optimization

Data preprocessing - **standard scaler**

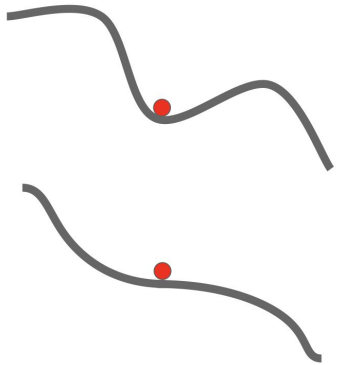
Reminder

- Generally good practice to **preprocess data to have mean 0, and standard deviation 1** (i.e. standardized)
  - Use standard scaler

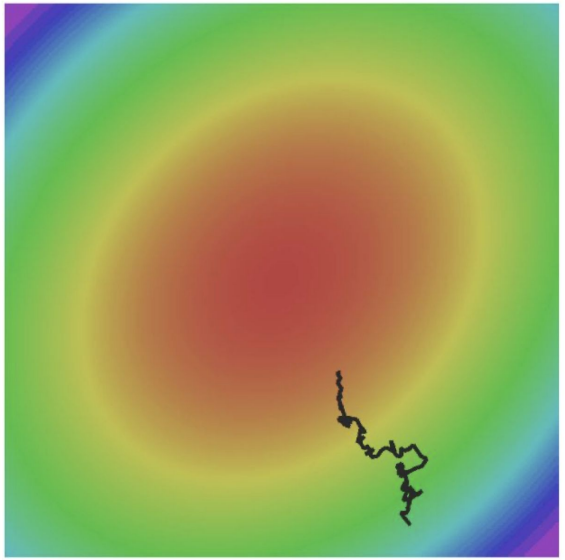


Problems with vanilla SGD

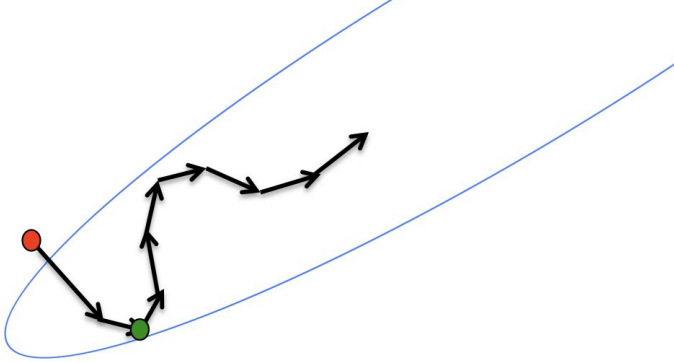
- What if the loss function has a local minima or saddle point?
  - Zero gradient → gradient descent gets stuck
- Saddle points **much more common in high dimensions**



- Moreover our gradients come from minibatches so **they can be noisy!**



Momentum



- Imagine a ball on the error surface
- If the ball has a mass, it starts off by following the gradient, **but once it has velocity, it no longer does the steepest descent**
  - Its momentum makes it keep going in the previous direction

- The net effect is that
  - it **damps oscillations in the directions of high curvature** by combining gradients with opposite signs
  - It **builds up speed** in directions with a gentle by consistent gradient

SGD + momentum update

SGD:

$$w_{t+1} = w_t - \eta \nabla L(w_t)$$

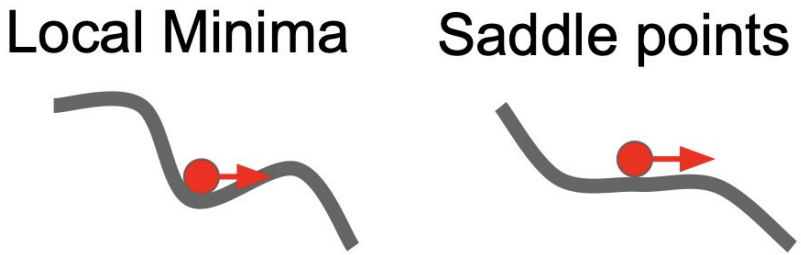
SGD + momentum:

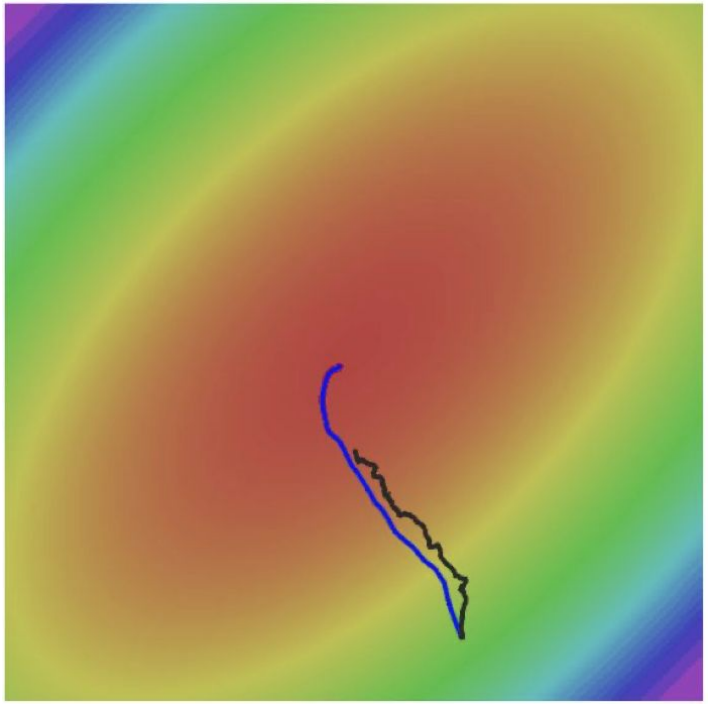
$$v_{t+1} = \rho v_t - \eta \nabla L(w_t)$$

$$w_{t+1} = w_t + v_{t+1}$$

- Build up “velocity” as a running mean of gradients
- Effect of the gradient is to increment the previous velocity
- Velocity also decays by ρ, usually 0.9 or 0.99
  - Can be thought of as “friction”
- The weight change is equal to the current velocity

SGD + momentum update





Optimizers with **adaptive learning rates**

- In a multilayer net, **the appropriate learning rates can vary widely between weights**
  - The magnitudes of the gradients are often very different for different layers
  - The fan-in of a unit determines the size of the overshoot effects caused by simultaneously changing many of the incoming weights of a unit to correct the same error
- → **Use a global learning rate** (set by hand) **multiplied by an appropriate local gain** determined empirically for each weight

- **AdaGrad** normalizes the learning rate per parameter based on the sum of squares of previous gradients

$$w_{t+1,i} = w_{t,i} - \frac{\eta}{\sqrt{G_{t,ii} + \epsilon}} g_{t,i} \qquad G_t = \sum_{\tau=1}^{t} g_\tau g_\tau^\intercal \qquad g_{t,i} = [\nabla L(w_t)]_i$$

Sum of squares of prev. grads

Gradient

- **RMSProp** and **AdaDelta** use a running average with a decay rate of γ (usually 0.9) instead of saving all previous gradients

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{E[g^2]_t + \epsilon}} g_t \qquad E[g^2]_t = \gamma E[g^2]_{t-1} + (1-\gamma) g_t^2$$

Adaptive momentum estimation: **Adam**

- **Adam combines many of the previous tricks**

- m_t and v_t estimate the 1st moment (**mean**) and 2nd moment (uncentered **variance**) of the gradients
- As m_t and v_t are initialized as 0s, they are biased towards 0, especially initially and when β_1 and β_2 are close to 1
- Counteract these biases by computing bias-corrected first and second moment estimates $\hat{m}_t$ and $\hat{v}_t$
- Typically $\beta_1 = 0.9$ and $\beta_2 = 0.999$ and $\epsilon = 10^{-8}$

- In short: **Adam "feels the ground"** with automatic transmission and adaptive cruise control ;)

gradients

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

Adam update:

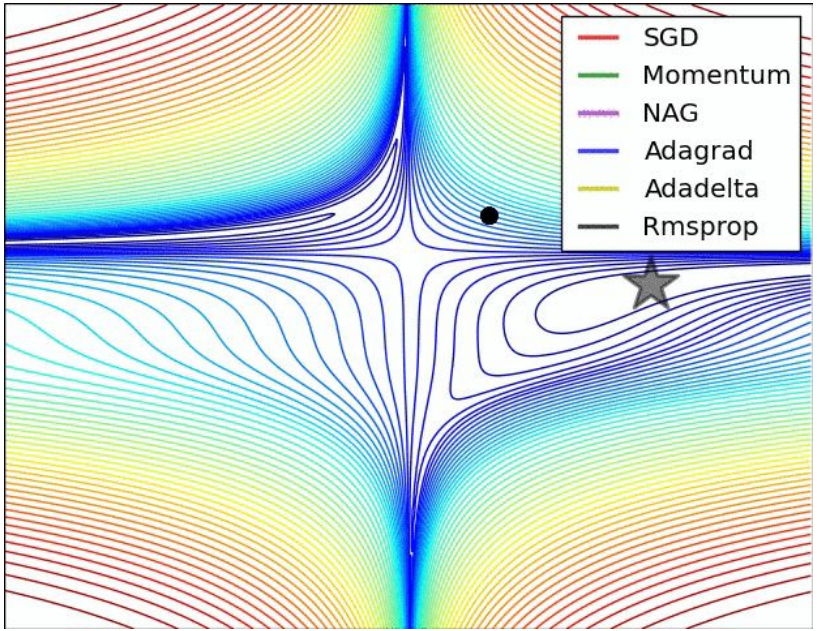
Momentum push

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

Scale (i.e. how fast to go)

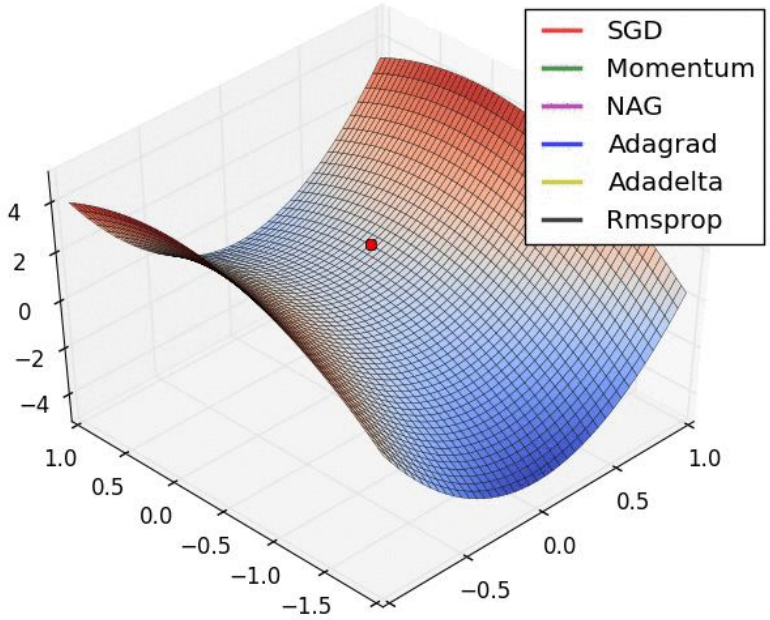
Visualization and **comparison of algorithms**

Optimization on loss surface contours



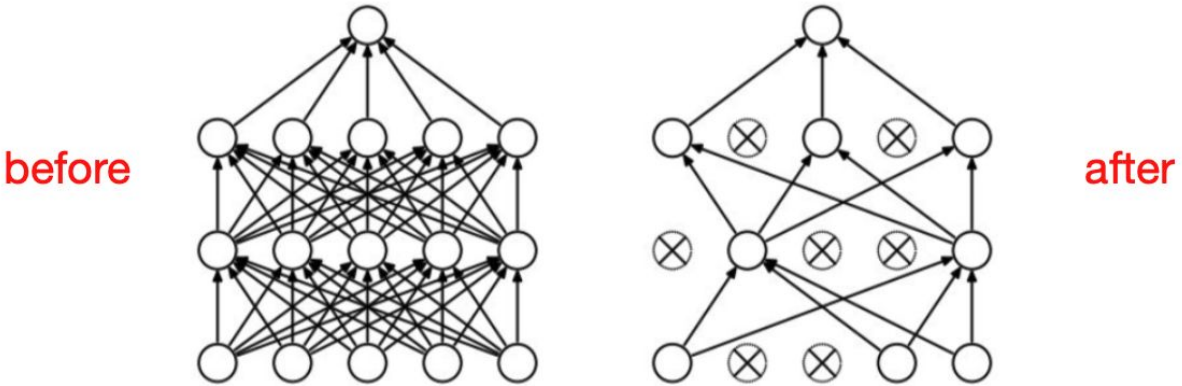
Credit: https://www.ruder.io/optimizing-gradient-descent

Optimization on saddle point



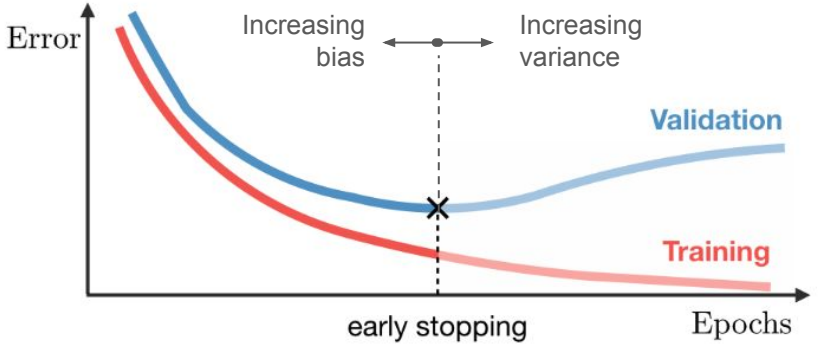
Dropout

- Powerful technique to **prevent overfitting in DNNs**
- It shuts down randomly a fraction of the connections
- **Forces the model not to rely excessively on particular sets of features**

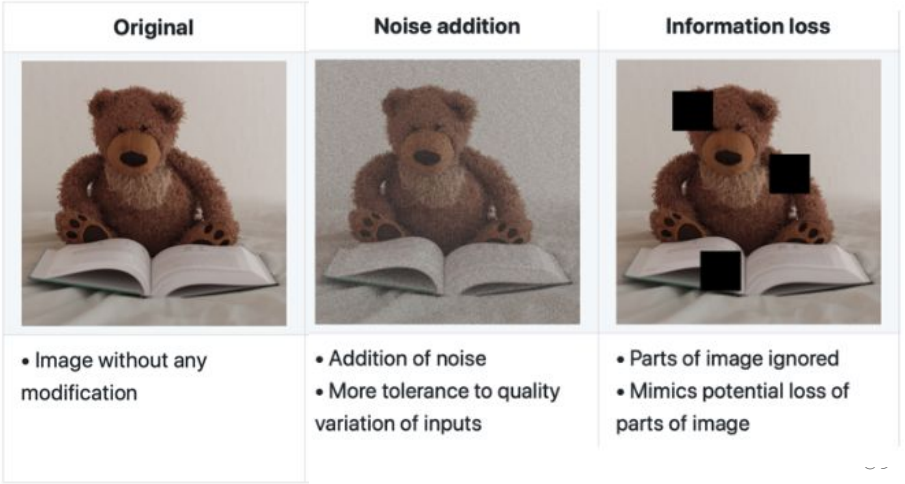


Early stopping and noise injection

- **Early stopping:** imposes constraints on the error reduction on the training set
  - The training process is stopped as soon as the loss on the validation sample reaches a plateau or start to increase



- **Noise injection/information loss:** makes it more difficult for the network to learn specific characteristics of the input features
  - random flip of labels
  - random occlusion of pixels or feature bits
  - adding withe/colored/gaussian noise to the features ...



Data augmentation

- One of the best ways to make an ML algorithm to generalize better is to **train it on larger and more expressive data**
- If the dataset size is limited → **artificially increase** the dimension of the training set by applying transformations that preserve the “physics” of the data/problem

Original	Flip	Rotation	Random crop	Color shift	Contrast change
					
• Image without any modification	• Flipped with respect to an axis for which the meaning of the image is preserved	• Rotation with a slight angle • Simulates incorrect horizon calibration	• Random focus on one part of the image • Several random crops can be done in a row	• Nuances of RGB is slightly changed • Captures noise that can occur with light exposure	• Luminosity changes • Controls difference in exposition due to time of day

Dealing with real data

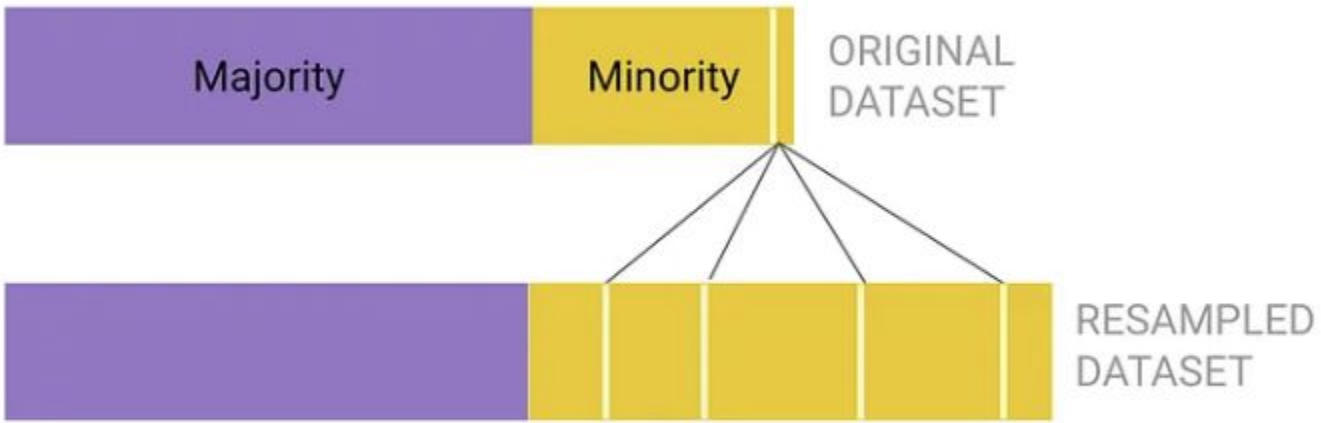
- James, cap.12.3

Real datasets are far from ideal - **missing and imbalanced data**

- Survey data
  - In favor: 30%, Opposed: 50%, The rest: Non-response!
- Longitudinal studies and clinical trials
  - Subject **drop** out or **censoring**
- Recommendation systems
  - Different individuals interact with or express preferences for different items
- Data integration
  - Different variables collected (with different frequencies) by different organizations or in **different experiments** or trials

Imbalanced dataset - **the resampling approach**

- A model trained on an imbalanced dataset **is expected to perform poorly on the minority class**
- Option 1: **oversample the minority class**
  - **High risk of overfitting**, the model learns the patterns of the minority class exactly



Imbalanced dataset - **the resampling approach**

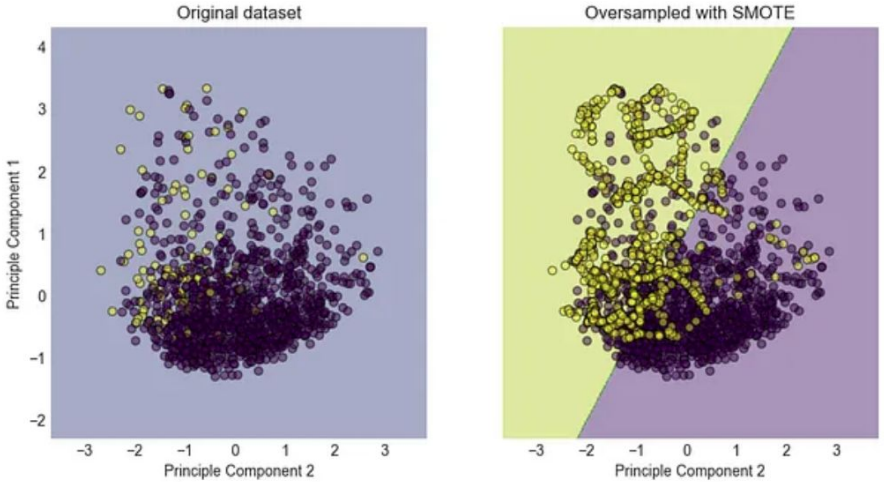
- A model trained on an imbalanced dataset **is expected to perform poorly on the minority class**

- Option 2: **undersample the majority class**
  - **Need a large dataset + potentially loose information**, the model becomes less precise in describing the classes



Imbalanced dataset - **resampling with SMOTE** (Synthetic Minority Over-sampling Technique)

- Intuitively, SMOTE **creates synthetic data points by interpolating between the minority data points that are close by to one another**
  - Randomly select some data points in the minority class
  - For every selected point, identify its nearest neighbour
  - For every neighbor, add a new point somewhere between the data point and the neighbor
  - Repeat steps 2 to 4 until sufficient synthetic data points are created
- Smaller risk of overfitting, may introduce some bias



Many more methods exist (beyond the scope of this discussion)

An alternative to resampling: **class weighting**

- During training, a model should learn two things:
  - **What each class looks like** → what feature values correspond to what class?
  - **How common each class is** → what is the relative distribution of the classes?
- Pure downsampling introduces a **prediction bias** by showing the model an artificial world where the classes are more balanced than in the real world

- In order to limit the bias:
  - Step1: **Downsample the majority class** → simply omit many of the majority class examples from training
  - Step2: **Upweight the downsampled class** → treat the loss on a majority class example more harshly than the loss on a minority class example

- **Downsampling + upweighting** results in:
  - **Better model**: The resultant model "knows" both of the following:
    - The connection between features and labels
    - The true distribution of the classes
  - **Faster convergence**: During training, the model sees the minority class more often, which helps the model converge faster.

$$Loss_{\text{total}} = \frac{1}{N}\sum(Loss_i \cdot w_{\text{class_}i})$$

The typical **supervised learning pipeline** for regression

●Data preparation

- Training dataset: $S = \{(x_1, y_1), \ldots, (x_N, y_N)\}$ where $x \in \mathbb{R}^D$ and $y \in \mathbb{R}$
- Model / hypothesis class: $f(x \mid w) = w^\intercal x$ (linear models)
- Loss function: $L(y, y') = (y - y')^2$ (squared loss)
- Optimization algorithm: SGD with regularization (L^1 or L^2)
- Cross validation and model selection:
- Testing and deployment

For regression

or $\phi(x)$ instead of x



Select λ

Example…

Let’s open the box and implement a NN **by hand**

- We use the **white wine quality dataset** from kaggle also available here
- The goal is to analyze the wine features (11) and predict (regress) the wine quality

data: (4898, 12)

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality
count	4898.000000	4898.000000	4898.000000	4898.000000	4898.000000	4898.000000	4898.000000	4898.000000	4898.000000	4898.000000	4898.000000	4898.000000
mean	6.854788	0.278241	0.334192	6.391415	0.045772	35.308085	138.360657	0.994027	3.188267	0.489847	10.514267	5.877909
std	0.843868	0.100795	0.121020	5.072058	0.021848	17.007137	42.498065	0.002991	0.151001	0.114126	1.230621	0.885639
min	3.800000	0.080000	0.000000	0.600000	0.009000	2.000000	9.000000	0.987110	2.720000	0.220000	8.000000	3.000000
25%	6.300000	0.210000	0.270000	1.700000	0.036000	23.000000	108.000000	0.991723	3.090000	0.410000	9.500000	5.000000
50%	6.800000	0.260000	0.320000	5.200000	0.043000	34.000000	134.000000	0.993740	3.180000	0.470000	10.400000	6.000000
75%	7.300000	0.320000	0.390000	9.900000	0.050000	46.000000	167.000000	0.996100	3.280000	0.550000	11.400000	6.000000
max	14.200000	1.100000	1.660000	65.800000	0.346000	289.000000	440.000000	1.038980	3.820000	1.080000	14.200000	9.000000

- Now that we’ve see the full chain, let’s implement a simple FFNN by hand
  - One hidden layer with 50 nodes
  - Let’s compare with the results from a similar implementation in Keras